REG N-⁰:223026681

UNIVERSITY OF RWANDA

COLLEGE OF SCIENCE AND TECHNOLOGY(CST)

GAKO CAMPUS

COMPUTER ENGINEERING

LEVEL III

MODULE: MOBILE APPLICATION

DATE: ON 28 MAR, 2026

ASSIGNMENT: LAB 4 CONSUMING **APIS IN FLUTTER**

Introduction

This project is a Flutter mobile application called "Posts Manager" developed to demonstrate API integration using a test API. The application allows users to perform CRUD operations including viewing, creating, editing, and deleting posts.

PART I: HAND WRITTEN SCANNED DOCUMEND

1. Question 1: Which dependencies did I use and Why?

Solution: dependencies Used

In This application, I used the HTTP dependency.
the http package is used to Connect the flutter application with the API. It allow the app different HTTP Methods
Include:

* GET: to fetch all posts
* Post: to create a new post
* PUT: to Update an existing post
* DEleta: to remove a post

this HTT dependency enables Communication between the mobile application and the backed API

2. HANDLING API EXCeptions

When Working with APIs, different types of errors may rise such as network failures or Invalid responses.
To handle these errors, the following method used:

* Try-Catch Block:
this is used to Catch errors When an API request fails

* Checking Status Codes:
The response Status Code is checked to Confirm if the request Was Successful (e.g. 200 for Success).

* Timeout Handling: A timeout Can be set to prevent the app for Waiting too long for a response.
So this error handling methods if error occurs, the application shows an error message instead of crashing.
this Improve the Stability and User experience of the application.

3. Future Widget Explanation

- A Future in flutter represents a Value that will be available later after a Synchronous operation is Completed.
- In this app, data is fetched from an API which takes time so future is used
- The FutureBuilder widget is used to build the user interface based on the result of the Future

It Works in three main States:

* Loading State:
When data is being fetched, a loading indicator is shown
* Success State: When data is received Successfully, it displayed on the Screen using a List View.
* Error State: if the error occurs, an error message is displayed

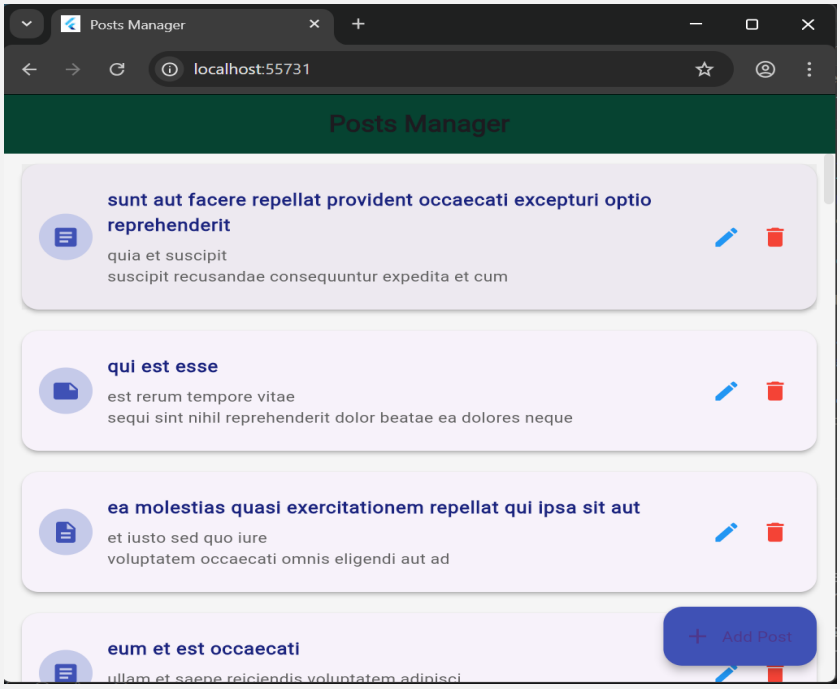
In this app, Future Builder is used on the home Screen to display the list of posts retrieved from the API.
this ensure that the UI Updates automatically When data is available.

Conclusion

This application demonstrates how Flutter Can be used to Consume APIs and perform CRUD operation. It also shows the importance of handling asynchronous data and errors to build reliable and Well Structured Mobile applications

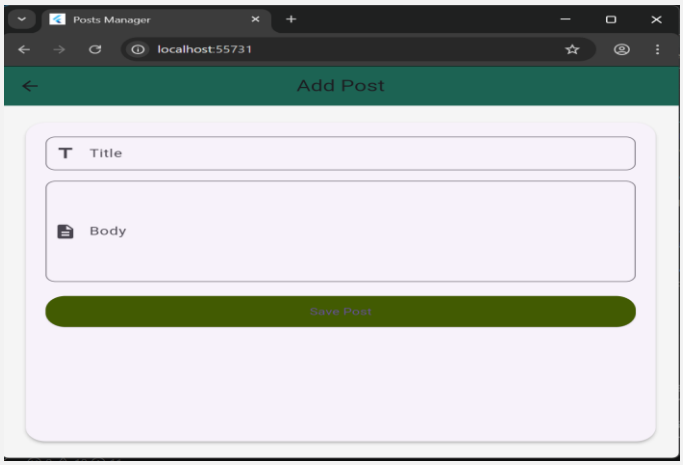
PART II: SCREEN SHOT OF PAGES WITH FEW EXPLAINATION

1. HOMEPAGE SCREEN SHOT(Posts List)



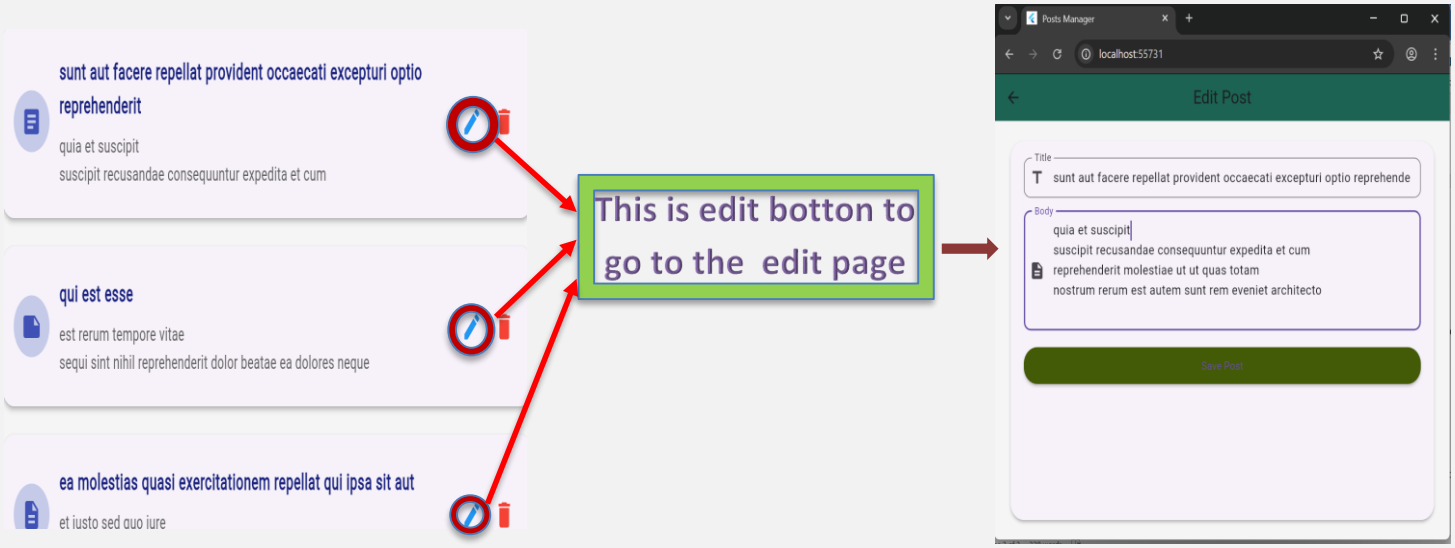
This screen displays all posts retrieved from the API. The data is loaded using FutureBuilder and shown in a scrollable list. Each post includes options to view details, edit, or delete.

2.ADD POST SCREEN SHOT



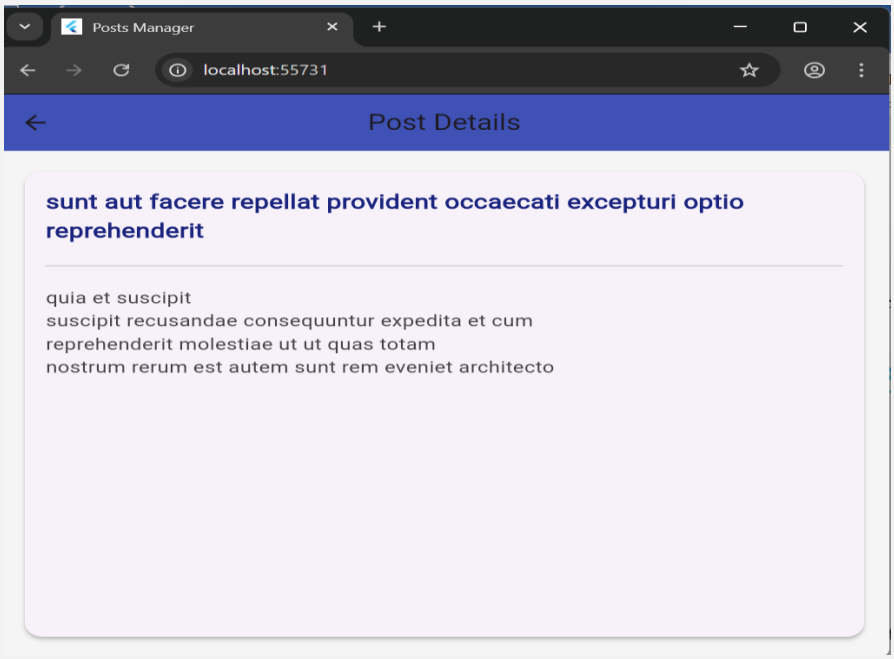
This screen allows the user to create a new post by entering a title and body. The data is then sent to the API using a POST request

3. EDIT POST SCREEN SHOT



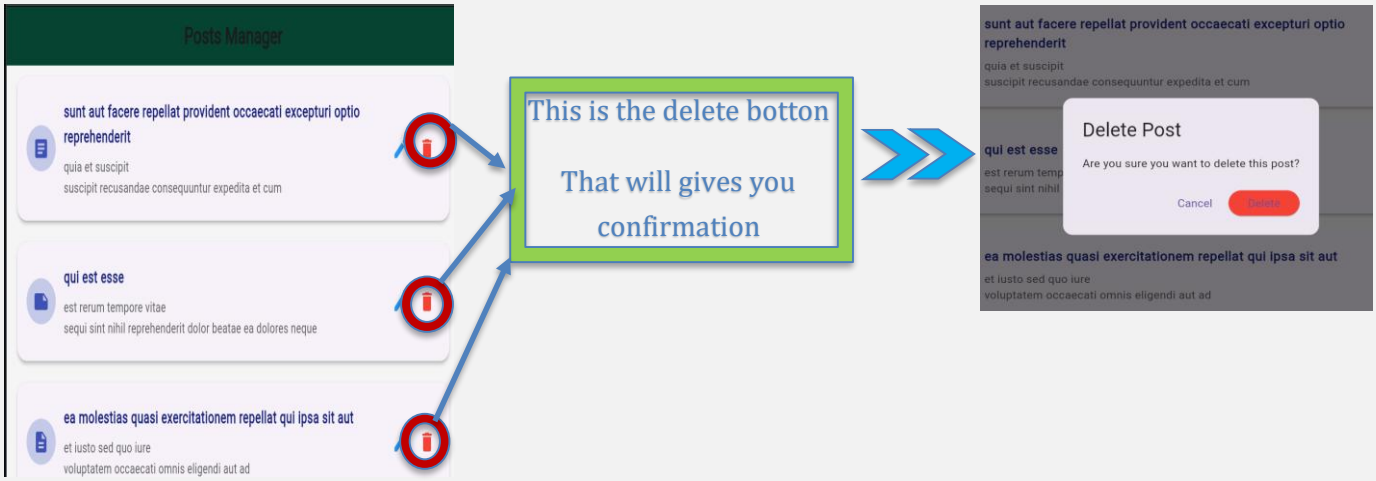
This screen allows the user to update an existing post. The form is pre-filled with existing data and updates are sent to the API using a PUT request.

4. POST DETAIL SCREEN



This screen shows the full details of a selected post, including its title and body content.

5. DELETE CONFIRMATION POP-UP



This dialog asks the user to confirm before deleting a post. It helps prevent accidental deletion and improves user experience.

Conclusion

This project demonstrates how a Flutter application can interact with an API to perform basic CRUD operations such as creating, reading, updating, and deleting posts. It shows how data can be fetched and displayed using the FutureBuilder widget and how user actions are connected to API requests.

The application also highlights the importance of handling errors and providing feedback to users for a better experience. Overall, this project provides a clear understanding of API integration in Flutter and how to build a simple and functional mobile application.

End